

```

/* cspell:disable */ import { config } from "./gulp.config.js"; import gulp from "gulp";

// Импорты инфраструктуры (Оставляем строго легкие утилиты контроля среды) import { isProd } from
"./gulp/server.js"; import { lintCss, lintJs } from "./gulp/lint.js"; import { cleandist, zipFiles, buildcopy } from
"./gulp/utls.js"; import { getBuildSignature } from "./gulp/system/gulp.cache.js";

// Инструменты автоматизации CLI (Перенаправлены в изолированную папку gulp/system/) import {
create } from "./gulp/system/gulp.create.js"; import { createModule as module } from
"./gulp/system/gulp.module.js"; import { remove } from "./gulp/system/gulp.remove.js"; import {
createStructure as init } from "./gulp/system/gulp.init.js"; import { help } from "./gulp/system/gulp.help.js";

const version = getBuildSignature(); console.log( 📦 [CONTROL]: Сборка Первого шаблона выполняется
под сигнатурой: ${version}, );

const { parallel, series } = gulp; const loadedModules = {};

// 🌀 КАРТА ДЕКЛАРАТИВНОГО МАППИНГА ЗАДАЧ (ДЛЯ СВЕРХБЫСТРОГО LAZY LOADING) const
TASK_FILE_MAP = { styles: "styles", cssPurge: "styles", // Направляем очистку стилей в файл styles.js scripts:
"scripts", html: "html", fonts: "fonts", fontsStyle: "fonts", images: "images", imagesDev: "images", createWebp:
"images", sprite: "images", favs: "images", browsersync: "server", startwatch: "server", };

/**

  • Динамический загрузчик изолированных Gulp-модулей (Lazy Loading)

  • Предотвращает перегрузку оперативной памяти при старте CLI */ const runTask = (taskName) => {
const gulpTaskWrapper = async (done) => { try { const fileName = TASK_FILE_MAP[taskName] ||
taskName;

if (!loadedModules[fileName]) { loadedModules[fileName] = await import(`./gulp/${fileName}.js`); }

const taskModule = loadedModules[fileName]; const task = taskModule[taskName] ||
taskModule.default;

if (typeof task === "function") return task(done);

throw new Error( Экспортируемая функция "${taskName}" не найдена в файле
"./gulp/${fileName}.js", ); } catch (err) { console.error(\x1b[31m[Task Error] ${taskName}:
${err.message}\x1b[0m); done(err); } };

Object.defineProperty(gulpTaskWrapper, "name", { value: taskName }); return gulpTaskWrapper; };

// Комплексные агрегаторы ресурсов разработки const compileAssetsDev = parallel( runTask("styles"),
runTask("scripts"), runTask("html"), runTask("imagesDev"), runTask("createWebp"), runTask("sprite"), );

const compileAssetsProd = parallel( runTask("styles"), runTask("scripts"), runTask("images"),
runTask("createWebp"), runTask("sprite"), runTask("favs"), );

//
=====
== // ПОЛНЫЕ РЕЖИМЫ СБОРКИ ПРОЕКТА (BUILD & DEVELOPMENT PRODUCTION) ==

```

```
=====
==
```

```
// ПОЛНЫЙ ЦИКЛ СБОРКИ ДЛЯ ПРОДАКШЕНА (СТРОГАЯ ОЧЕРЕДНОСТЬ ПОТОКОВ) export const build =
series( cleandist, parallel( ...(isProd ? [lintCss, lintJs] : []), runTask("fonts"), runTask("fontsStyle"), ),
compileAssetsProd, runTask("html"), // 1. Сначала полностью собираем HTML (с вашими новыми )
buildcopy, // 2. Копируем остальные файлы runTask("cssPurge"), // 3. Чистим CSS на основе готового
HTML (строго ПОСЛЕ html) zipFiles, // 4. Архивируем результат (строго ПОСЛЕ очистки стилей) (done) =>
{ console.log(">>> 🚀 Project successfully assembled and archived! <<<"); done(); }, );
```

```
// СТАРТОВЫЙ TASK ДЛЯ ЛОКАЛЬНОЙ РАЗРАБОТКИ (Команда: npx gulp или npm run dev) export default
series( help, parallel(runTask("fonts"), runTask("fontsStyle")), compileAssetsDev, buildcopy,
runTask("browsersync"), runTask("startwatch"), );
```

```
// ЕДИНЫЙ ИЗОЛИРОВАННЫЙ БЛОК CLI-ЭКСПОРТА export { create, remove, module, init, help, cleandist,
lintJs, lintCss };
```

```
// Явная ленивая регистрация деструктурированных ссылок задач export const favs = runTask("favs");
export const styles = runTask("styles"); export const cssPurge = runTask("cssPurge"); export const scripts =
runTask("scripts"); export const html = runTask("html"); export const images = runTask("images"); export const
createWebp = runTask("createWebp"); export const sprite = runTask("sprite"); export const fonts =
runTask("fonts"); export const fontsStyle = runTask("fontsStyle"); export const browsersync =
runTask("browsersync"); export const startwatch = runTask("startwatch");
```